CSCI4130 Introduction to Cyber Security

Ryan Chan

March 18, 2026

Abstract

This is a note for **CSCI4130 Introduction to Cyber Security**.
Contents are adapted from the lecture notes of CSCI4130, prepared by Wei Meng, as well as some online resources.
This note is intended solely as a study aid. While I have done my best to ensure the accuracy of the content, I do not take responsibility for any errors or inaccuracies that may be present. Please use the material thoughtfully and at your own discretion.
If you believe any part of this content infringes on copyright, feel free to contact me, and I will address it promptly.
Mistakes might be found. So please feel free to point out any mistakes.

Contents

1 Introduction **2**

2 Control Hijacking **3**
2.1 Attack 3
2.2 Defenses 7

3 Cryptography **10**

Chapter 1

Introduction

Cyber Security is about keeping computing systems working as intended, keeping data accessed only as desired, and keeping the computing resources secured. All of these need to be done in the presence of an adversary and on a budget. We first introduce some concepts in this chapter.

Threat Model

A threat is a potential violation of security by an attacker. Since it is not possible to protect against all possible attackers, we need risk management. Computer security is always related to a specific threat model.

Threat modelling works to identify, communicate and understand threats and mitigations within the context of protecting something of value. A threat model includes:

- What to protect
- From whom to protect
- Assumptions (can be wrong)
- A list of the (most important) relevant threats
- What actions are to be taken for each threat

Security Policy

This is a statement on what is and what is not allowed.

Security Mechanism

This is a method, tool or procedure for enforcing a security policy. A security policy can be implemented by a variety of security mechanisms. For a security mechanism to work,

1. the security policy correctly describes the security requirement of a system
2. the security mechanism is implemented correctly
3. the security mechanism is deployed correctly
4. the security mechanism cannot be bypassed

Prevention

Prevention aims to prevent something bad from happening.

Detection

Detection is about knowing that something is wrong. There are two special cases, which are false positive, meaning this is a wrong alarm, and FALSE NEGATIVE, meaning there is an unnoticed attack.

Response

Response is needed when we detect something is going wrong.

Mitigation

Mitigation can be more economical and practical than prevention.

Chapter 2

Control Hijacking

Application software is computer software that performs a group of pre-defined functions or tasks. It can provide services locally or remotely, and its behavior is determined by the data/input being processed, the code being executed, and the environment in which it runs.

Compromising the security properties, i.e. **Confidentiality**, **Integrity**, and **Availability**, is the goal of attackers. But how can they achieve this?

2.1 Attack

Control hijacking attacks are a class of attacks that hijack the control flow of an application. Since application code is loaded into memory, attackers may corrupt memory through techniques such as buffer overflows, integer overflows, and format string exploitation. Before diving into the details, we first look at several related concepts.

2.1.1 Control Flow Graph

Applications may invoke external commands to perform specific tasks. For example, `system(<string>)` executes a command specified in a string by calling `/bin/sh -c <string>`. Therefore, once users control the string content, they can inject arbitrary commands.

It is helpful to understand the behavior of an application, i.e. its control flow. This describes the order in which code is executed or interpreted, and it can represent the functionality or behavior of a program to some extent.

We can use a **Control Flow Graph (CFG)** to graphically represent the control flow of a program. It shows all possible paths that might be traversed during the execution of the program.

A Control Flow Graph contains nodes, which are blocks of code without branches, connected by directed edges. A CFG should have an entry node and an exit node. A path is a sequence of nodes connected by edges. Note that not all paths are feasible.

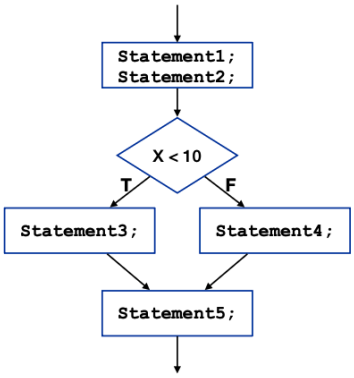

Figure 2.1: Control Flow Graph

2.1.2 Program State

A program state can be described by the values of processor registers and memory, such as the stack and heap.

Processor registers provide several quickly accessible storage locations. They hold data used in arithmetic and logical operations, and data is copied between registers and main memory as needed.

For memory, the stack usually grows downward (toward lower memory addresses). The stack pointer `ESP` points to the top of the stack. The `push` instruction decrements `ESP` and stores a value at the address pointed to by `ESP`. The `pop` instruction retrieves the value at the top of the stack into a register and then increments `ESP`.

The stack is composed of frames. Frames are pushed onto or popped from the stack when a function is invoked or returns. Each frame contains the function's actual parameters, the return address, a pointer to the previous frame, and the function's local variables (as shown in Figure 2.2). The stack base pointer `EBP` stores the address of the current frame.

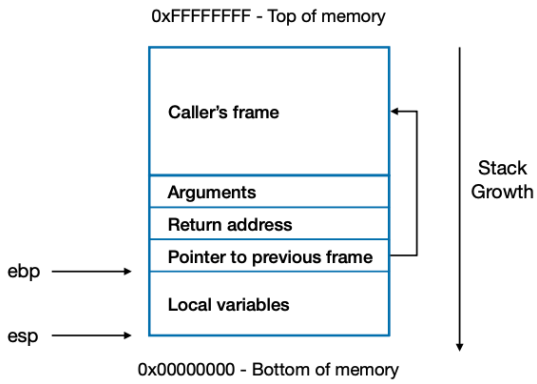

Figure 2.2: Stack Frame Layout

A simple walkthrough of a function call can be illustrated using the following example:

```
#include <ctype.h> // tolower
#include <string.h> // strcmp
#include <stdio.h> // fgets, fputs

void reveal_secret() {
  fputs("SUPER SECRET = 42\n", stdout);
}

int verify(const char* name) {
  char user[256];
  int i;
  for (i = 0; name[i] != '\0'; ++i)
    user[i] = tolower(name[i]);
  user[i] = '\0';
  return strcmp(user, "xyzzy") == 0;
}

int main() {
  char login[512];
  fgets(login, 512, stdin);
  if (! verify(login))
    return 1;
  reveal_secret();
  return 0;
}
```

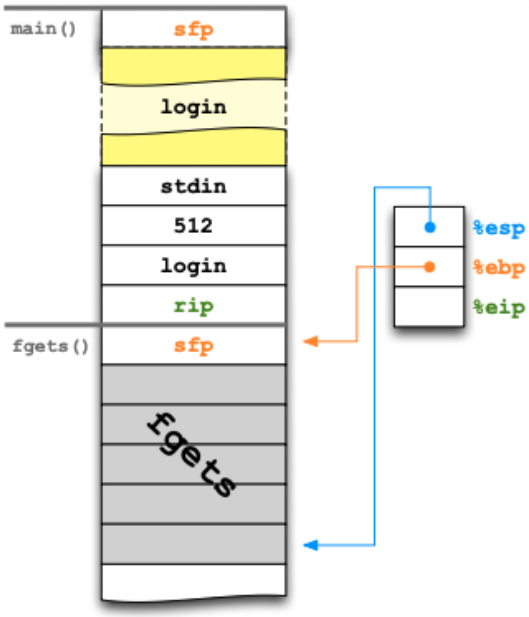

Figure 2.3: Stack - Function Call

Consider the entry point of this program at `int main()`. The saved frame pointer (SFP) points to the old frame pointer (OFP), which is pushed onto the stack, causing `ESP` to decrement. Since this is now the current frame, `EBP` will point to this position and will remain unchanged unless another function is called.

Next, `login[512]` is allocated on the stack. The program then reaches the first function call, `fgets`. The arguments of the function are pushed onto the stack in reverse order, i.e. `stdin`, `512`, and then `login`. After that, the return instruction pointer (RIP) is pushed onto the stack so that the program knows where to resume execution after the function call returns.

These steps repeat for subsequent function calls until the program finishes executing all function calls.

2.1.3 Overflow

Overflow is one of the most popular classes of attacks, as it is exploitable both locally and remotely, and it can modify both the data and the control flow of an application.

More specifically, we will discuss stack overflow. A function's local variables (buffers) are statically allocated on the stack, and data may be copied or written without boundary checking. Therefore, data can overflow a buffer and overwrite other important data, e.g. the return address or the saved `EBP`. With a carefully crafted input, it is possible to overwrite security-sensitive data, redirect execution to user-defined code, or even reuse existing code. We will examine three cases.

Overflow – Crash

Consider the example above. Suppose the user inputs a value of 500 bytes. After the `fgets` function call finishes, the program proceeds to the `verify(login)` function call. Inside `int verify`, the program iterates through the input and calls `tolower`. The resulting characters are then stored in the array `user[256]`.

Since the length of `name` is 500, writing the data into `user[256]` causes the buffer to overflow. As a result, the content beyond `user[256]` overwrites other data on the stack, including the saved frame pointer and the return address (instruction pointer / program counter, `EIP`). When the function returns, the program attempts to jump to the corrupted return address. This typically redirects execution to an invalid or unknown address, causing the program to crash.

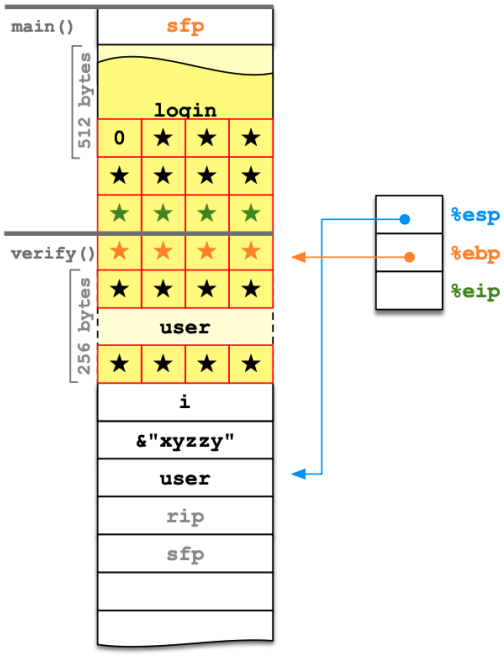

Figure 2.4: Stack Overflow - Crash

Overflow – Inject

Assume the attacker provides input containing machine code (shellcode), e.g.

```
a0c2d782
ffa86db2
307abba9
ad7c....
```

When this input is processed by the program (`user[i] = tolower(name[i])`), the bytes are copied into the buffer `user`. If the input exceeds the buffer size, a carefully crafted overflow can overwrite the saved frame pointer (SFP) and the saved return address (RIP). By replacing the saved return address with the address of the injected shellcode in the buffer, the attacker can cause the program to jump to the shellcode when the function returns. The shellcode can then execute a system call such as `execve("/bin/sh")` to spawn a shell.

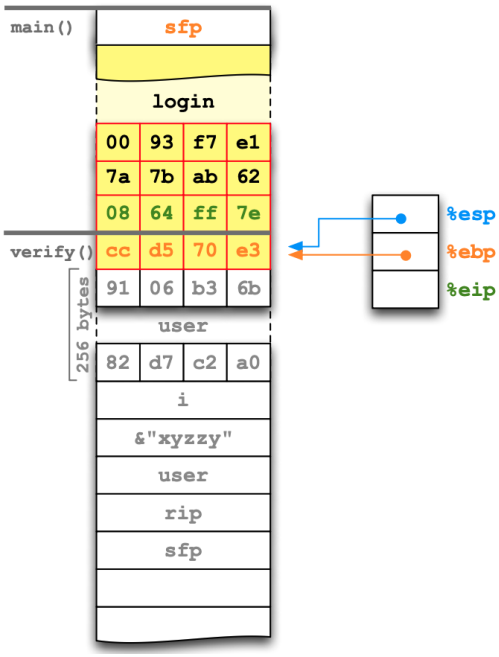

Figure 2.5: Stack Overflow - Inject

Overflow - Divert

This type of overflow is similar to the code injection. Assume the address of `reveal_secret` is `0xc8f4f2a9`. Similar to the previous case, with a carefully crafted overflow, the user can overwrite the return instruction pointer (RIP) with `0xc8f4f2a9`. Then, when the function call ends, the program will jump to `reveal_secret` and display the message 'SUPER SECRET = 42'.

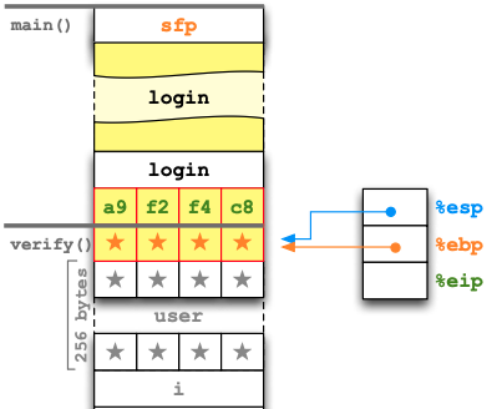

Figure 2.6: Stack Overflow - Divert

There are several other vulnerable functions in C/C++, e.g. `fgets()`, `strcpy()`, `strcat()`, `sprintf()`, `vsprintf()`, `scanf()`, `sscanf()`, etc. Any reference to a value that can be overwritten may represent a security vulnerability. For example, we can overwrite the following:

1. Data and data pointers
2. Return address
3. Stack frame pointer
4. Exception handler
5. Function pointer
6. Jump destination

Therefore, safer versions of memory access functions should be used to ensure that important data structures, i.e. the stack, heap, and others, cannot be overwritten. However, there are still some possible overflows.

Integer Overflow

Integer overflow is caused by unexpected results when comparing, casting, or performing arithmetic operations on integers. There are several cases:

1. A large signed integer can become a negative value.
2. A large unsigned integer can become a small unsigned integer.
3. A negative signed integer can become a large unsigned integer.

Therefore, integers of different sizes or types should be assigned with great care, and assertions or similar checks can be used to avoid overflows.

Index Overflow

Index overflow exploits the lack of boundary checks when referencing a value in an array using an index. This allows direct writes to memory locations. Depending on the situation, it may only be possible to modify memory values that conform to the type of elements in the array.

For example, consider an array declared as `array[10]`. If the program reads input without checking the bounds and writes to a position beyond indices 0 to 9, it causes an overflow.

> **Remark.** In C++, objects can be dynamically allocated on the heap. Each object is associated with a pointer pointing to the virtual function table (vtable), which is a table of pointers to the definitions of the object's methods. A variable overflow can be used to overwrite this pointer so that it points to a fake virtual function table specified by the attacker.

The goal of attackers who attempt to corrupt the memory of a process is usually to take control of it such that they can run with the privileges of the process and execute attacker-specified code. These attacks are made possible by mixing data and control in memory and allowing users to overwrite control data.

Therefore, we need to look at the defenses against such attacks.

2.2 Defenses

We want to make sure that for memory accesses, including read, write, and execute, there is no access to 'undefined' memory. Note that 'undefined' is with respect to the semantics of the programming language. To make sure the code executes safely, we utilize a module-by-module analysis.

2.2.1 Conditions

Here we discuss two conditions: preconditions and postconditions.

Preconditions describe what must hold for a function or statement to operate correctly. Formally, a precondition is an assertion that must hold about the inputs of a function for it to work correctly. The caller must satisfy the precondition, and the callee may assume that the precondition is met.

Postconditions describe what holds after the function returns or the statement is executed. A postcondition is an assertion that must hold after a function returns. The caller may assume that the postcondition holds, while the callee must ensure that the postcondition holds. For consecutive statements, the postcondition of statement 1 should logically imply the precondition of statement 2.

There are also **invariants**, which are conditions that always hold at a given point in a function.

For example, consider

```
int deref(int *p) {
  return *p;
}
```

The precondition that must hold is that the pointer `p` is not `NULL` and is valid. For

```
void *my_malloc(size_t n) {
  void *p = malloc(n);
  if (!p) {
    perror("malloc");
    exit(1);
  }
  return p;
}
```

The postcondition here is that the return value is not `NULL` and `p` is a valid pointer.

In general, the correctness proof strategy for memory safety is to identify each memory access point, write down the precondition it requires, and propagate the requirement up to the beginning of the function.

In cases with more complicated loops, induction might be used. The base case corresponds to the first entrance into the loop, and the induction step shows that the postcondition of the last statement in the loop, together with the loop test condition, implies the invariant.

2.2.2 Approaches

To defend against attacks due to memory corruption, we have two approaches.

Prevention

We aim to write bug-free programs, or rewrite programs in a memory-safe language that automatically checks boundaries and does not allow unsafe pointer arithmetic. We can also analyze programs for potential vulnerabilities before execution, prevent attack code execution, and make exploitation harder.

Detection

We detect attacks by performing security checks during program execution, such as detecting abnormal sequences of executed instructions and checking for integrity violations.

We now look at four types of defense.

2.2.3 Canary

Canaries are placed in stack frames to prevent and detect overwriting of the return address on the stack. StackGuard inserts a canary before the return address in each stack frame, such that when the function returns, the canary value is verified against the stored value. This acts as a runtime check for stack integrity.

There are different types of canaries:

1. **Terminator Canary**

This type of canary uses `NULL` (`0x00`), `CR` (`0x0d`), `LF` (`0x0a`), and `EOF` (`0xff`). String functions will not copy beyond a terminator, so attackers cannot use such functions to overwrite the return address.

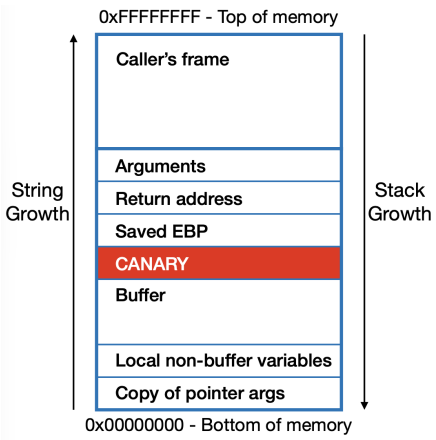

Figure 2.7: Stack with canary

However, terminator canaries are not completely random. For example, there are always fixed zero bytes, making it easier to guess the canary and potentially brute-force it.

2. **Random Canary**

A random value is chosen at program startup and stored in a location known only to the validation code, often protected using unmapped pages.

3. **XOR Canary**

This uses `random value` $\oplus$ `return address`.

There are also limitations to canaries. Canaries do not provide full protection because they can be learned. For example, there might be format string vulnerabilities or information leakage, and canaries can also be overwritten. Moreover, other pointers can still be overwritten. For example, a function pointer in the stack frame can be overwritten.

Canaries can also be bypassed by overflowing a pointer used as the destination of a `strcpy()`-like function. Some stack-smashing techniques can overwrite the return address without touching the canary (note that the XOR canary was introduced to protect against this attack). Also, enabling canary protection requires recompiling the program, which can be expensive.

2.2.4 Non-Executable Memory

In order to prevent attacker-supplied code from executing, the operating system can mark the stack and heap as non-executable.

> **Remark.** Linux leverages the NX (No-eXecute) bit in the page table entry to mark certain memory areas as non-executable. Windows implements this mechanism using Data Execution Prevention (DEP), which can emulate the mechanism if the NX bit is not supported by the processor. OpenBSD implements W $\oplus$ X, which enforces that a page is either writable or executable, but not both.

These mechanisms aim to ensure that no memory region is both writable and executable. However, some applications may require writable memory that is also executable. In addition, attackers can still hijack the control flow of a vulnerable application even if the memory is non-executable, for example, via return-to-libc or return-oriented programming (ROP).

If the memory is non-executable, attackers cannot launch code-injection attacks. However, they can

still hijack the control flow by using existing functions or code snippets in libraries and chaining them together to perform the desired tasks.

2.2.5 Address Space Layout Randomization (ASLR)

This defense maps the stack, heap, code, and shared libraries to random locations in the process memory. The library code needs to be position-independent; otherwise, runtime mapping may incur overhead. This makes jumping to a library function more difficult.

In Linux, ASLR is enabled by default. It is implemented by the kernel in collaboration with the ELF loader.

To exploit ASLR, brute-forcing can be used if the entropy is low, i.e. if it is possible to limit the variation space.

> **Remark.** In security, entropy refers to how many possible values something can take. High entropy means many possibilities, making it hard to guess. Low entropy means fewer possibilities, making it easier to guess via brute-force.

2.2.6 Control Flow Integrity

Previously, we discussed the control flow graph (CFG). Programs have a CFG, and control-hijacking attacks usually result in execution paths that do not exist in the CFG. Therefore, control flow integrity (CFI) ensures that the application follows only the control flows specified by the CFG.

To enforce CFI, the CFG is built at compile time, and the application is instrumented to check at runtime the validity of each control transfer.

For example, in Windows 10, Control Flow Guard (CFG) prevents jumping to invalid locations through indirect calls by checking against a bitmap of all valid function entry points in the executable. However, it does not prevent an attacker from jumping to a valid but unintended function, so it is difficult to statically build a fully accurate CFG.

Chapter 3

Cryptography